Lab 3: Distance, Dimensionality, and Why Geometry Breaks

Learning Objectives

By the end of this lab, you should be able to:

- Explain why distance becomes less informative as dimensionality increases
- Interpret numerical distance outputs instead of just computing them
- Describe the *curse of dimensionality* in plain language
- Understand why nearest neighbors stop being meaningfully "near"
- Connect geometric intuition to probabilistic reasoning using Chebyshev's inequality

This lab is intentionally conceptual. The code is simple; the interpretation is not.

1.4 Numerical Corner

In [1]:
```
# imports
import numpy as np
import matplotlib.pyplot as plt
from numpy import linalg as LA
import seaborn as sns
import plotly.express as px
import plotly.graph_objects as go
```

Determining k

In K-means clustering, one of the main challenges is determining the optimal number of clusters (k). In most real-world applications, (k) is not known in advance (usually either because of complex/high dimensional data or just not knowing the ground truth), and selecting an inappropriate value can lead to poor clustering results.

There are many ways to determine the best k - common ones are:

- Elbow methow
- Silhouette score
- Domain knowldege

In [2]:
```
# Functions we have seen before - K Means
def opt_reps(X, k, assign):
# our k-means functions are out_reps, opt_clust, and kmeans
    _n, _d = X.shape
    reps = np.zeros((k, _d))
    for _i in range(k):
        in_i = [_j for _j in range(_n) if assign[_j] == _i]
```

```
        reps[_i, :] = np.sum(X[in_i, :], axis=0) / len(in_i)
    return reps

def opt_clust(X, k, reps):
    _n, _d = X.shape
    dist = np.zeros(_n)
    assign = np.zeros(_n, dtype=int)
    for _j in range(_n):
        dist_to_i = np.array([LA.norm(X[_j, :] - reps[_i, :]) for _i in range(k)])
        assign[_j] = np.argmin(dist_to_i)
        dist[_j] = dist_to_i[assign[_j]]
    G = np.sum(dist ** 2)
    print(G)
    return assign

def kmeans(rng, X, k, maxiter=10):
    _n, _d = X.shape
    assign = rng.integers(0, k, _n)
    reps = np.zeros((k, _d), dtype=int)
    for iter in range(maxiter):
        reps = opt_reps(X, k, assign)
        assign = opt_clust(X, k, reps)
    return assign
```

In [3]:
```
# New Functions for Section 1.2.3
def spherical_gaussian(rng, d, n, mu, sig):
# (Section 1.2.3) A function to sample form a spherical gaussian distribution
    return mu + sig * rng.normal(0, 1, (n, d))

def gmm2spherical(rng, d, n, phi0, phi1, mu0, sig0, mu1, sig1):
# (Section 1.2.3) The code computes mixtures of spherical Gaussians, a special case
# It returns an d by n array X, where each row is a sample from a 2-component spher
    phi, mu, sig = (np.stack((phi0, phi1)), np.stack((mu0, mu1)), np.stack((sig0, s
    X = np.zeros((n, d))
    component = rng.choice(2, size=n, p=phi)
    for _i in range(n):
        X[_i, :] = spherical_gaussian(rng, d, 1, mu[component[_i], :], sig[componen
    return X
```

In [4]:
```
# New Function from Section 1.4.1
def two_mixed_clusters(rng, d, n, w):
    mu0 = np.hstack(([w], np.zeros(d - 1)))
    mu1 = np.hstack(([-w], np.zeros(d - 1)))
    return gmm2spherical(rng, d, n, 0.5, 0.5, mu0, 1, mu1, 1)
```

In [5]:
```
# Set seed and Initialize Random Number Generator
# Using a specific seed ensures reproducibility
seed = 535
rng = np.random.default_rng(seed)
```

In [6]:
```python
# Start with d = 2 (2 dimensions)
# TODO: generate the dataset X using two_mixed_clusters
# Hint: X should have shape (2*n, d)
_d, _n, _w = (2, 100, 3.0)
X_2d = two_mixed_clusters(rng, _d, 2 * _n, _w)
```

Why Distance Is the Problem

Most algorithms that rely on similarity (k-NN, clustering, kernels) assume:

> "Closer points are more similar than farther points."

This section shows why that assumption quietly fails as dimension increases.

Numerical Corner: This Is About Interpretation, Not Arithmetic

You are not being tested on whether you can compute a number.

You are being tested on whether you can explain:

- Why the number has this magnitude
- Why it changes with dimension
- Why your geometric intuition might be misleading

Question: Please explain what d, n, and w are?

Answer:

- **d**: Dimensionality (the number of features or columns in the dataset).
- **n**: The number of data points (samples) generated.
- **w**: Half-width or separation parameter. It represents the distance of the cluster centers from the origin along the first dimension (centers are at $+w$ and $-w$).

In [7]:
```python
# TODO: run kmeans on X using k=2
# Store the cluster assignments in `assign`
assign_2d = kmeans(rng, X_2d, 2)
```

```
1941.6019633105939
430.60862477322695
430.60862477322695
430.60862477322695
430.60862477322695
430.60862477322695
430.60862477322695
430.60862477322695
430.60862477322695
430.60862477322695
```

Question: Please explain what the output from the above code chunk is/means?

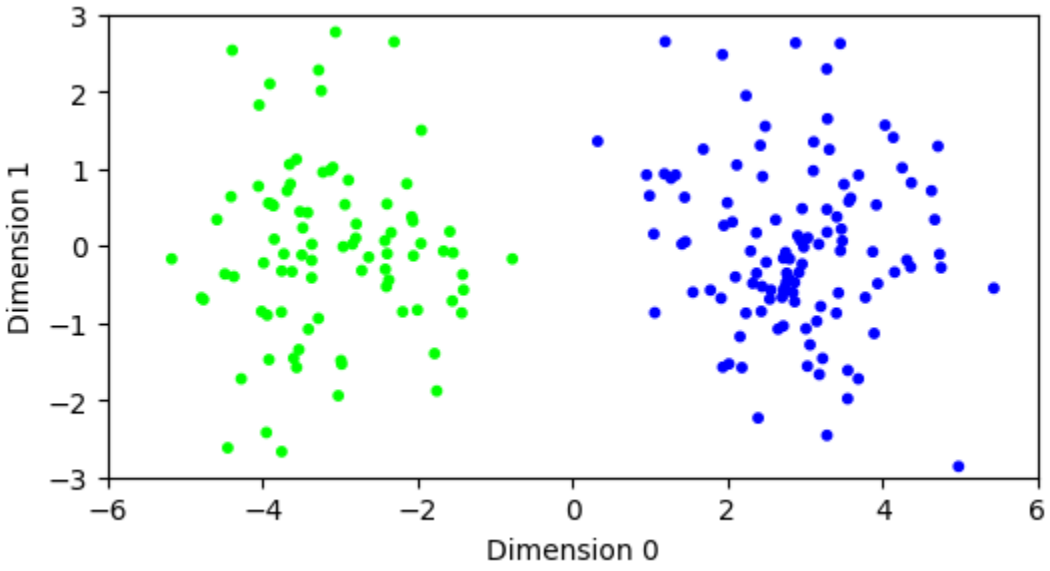
Answer: The output numbers printed represent the **Inertia** (or Sum of Squared Distances) for each iteration of the algorithm. As the algorithm iterates, the centroids move to better locations, minimizing the distance between points and their assigned centroids, causing this number to decrease until convergence.

Our default of 10 iterations seem to have been enough for the algorithm to converge (note that the default of maxiter is 10 in the above code).

In [8]:
```
X_2d.shape
```

Out[8]:
```
(200, 2)
```

In [9]:
```
# Visualize the result by coloring the points according to the assignment. (d = 2)
_i = 0
_j = 1
plt.figure(figsize=(6, 3))
plt.scatter(X_2d[:, _i], X_2d[:, _j], s=10, c=assign_2d, cmap='brg')
plt.axis([-6, 6, -3, 3])
plt.ylabel(f'Dimension {_j}')
plt.xlabel(f'Dimension {_i}')
plt.show()
```



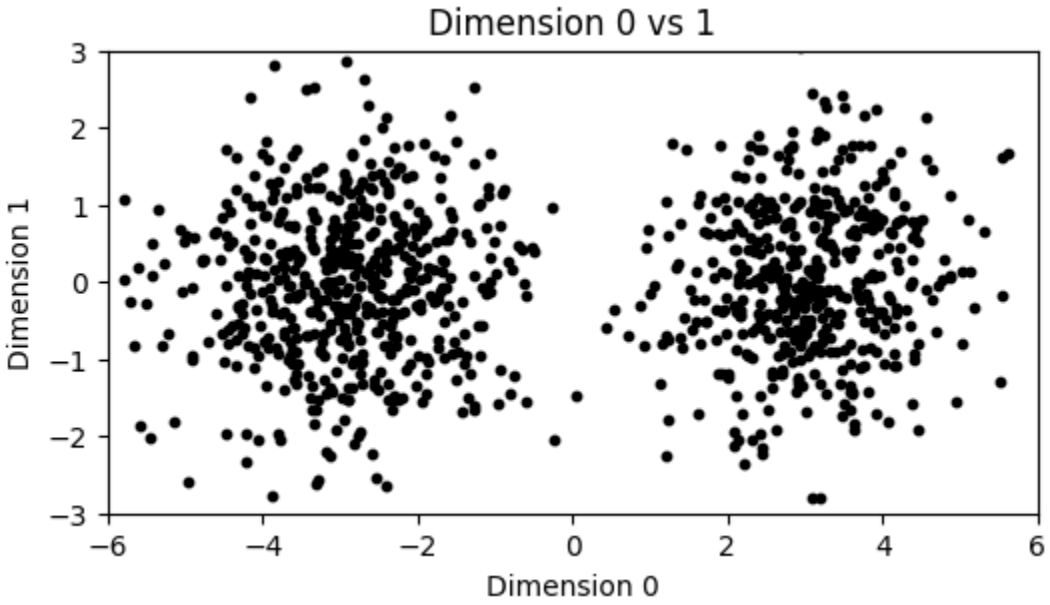
We see 2 clearly distinguishable clusters

Next, let's create a d = 1000 dimensional data set instead of just d = 2.

In [10]:
```
# Using d = 1000, plot the data in the first 2 dimensions with means centered at 3
# TODO: regenerate X using d = 1000
# Keep n and w the same
_d, _n, _w = (1000, 1000, 3.)
X_1000d = two_mixed_clusters(rng, _d, _n, _w)
X_1000d.shape
```

Out[10]:
```
(1000, 1000)
```

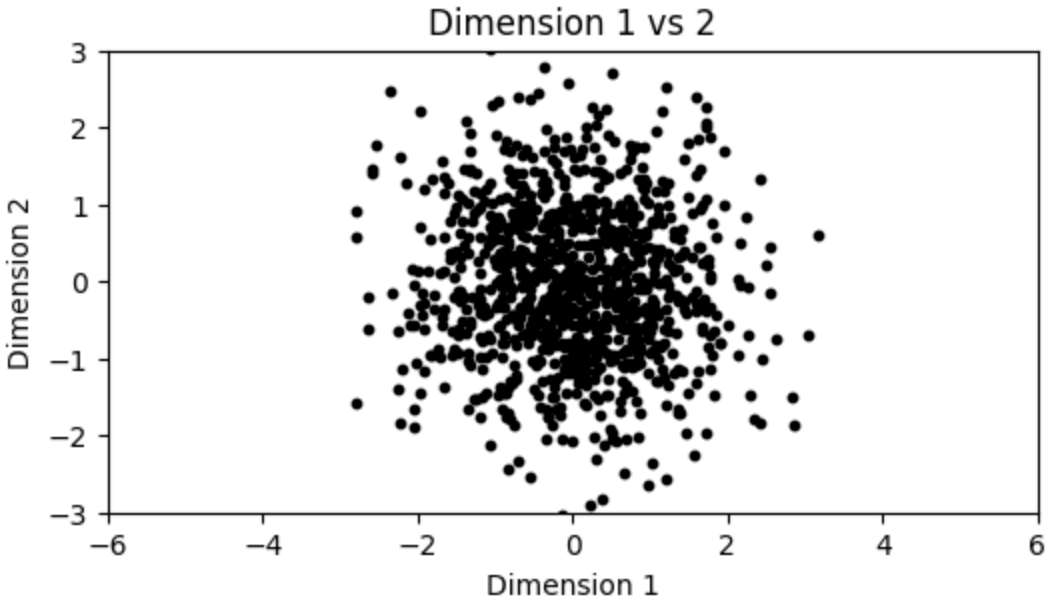
In [11]:
```
_i = 0
_j = 1
plt.figure(figsize=(6, 3))
plt.scatter(X_1000d[:, _i], X_1000d[:, _j], s=10, c='k')
plt.axis([-6, 6, -3, 3])
plt.ylabel(f'Dimension {_j}')
plt.xlabel(f'Dimension {_i}')
plt.title(f'Dimension {_i} vs {_j}')
plt.show()
```



We once again see two clearly distinguishable clusters.

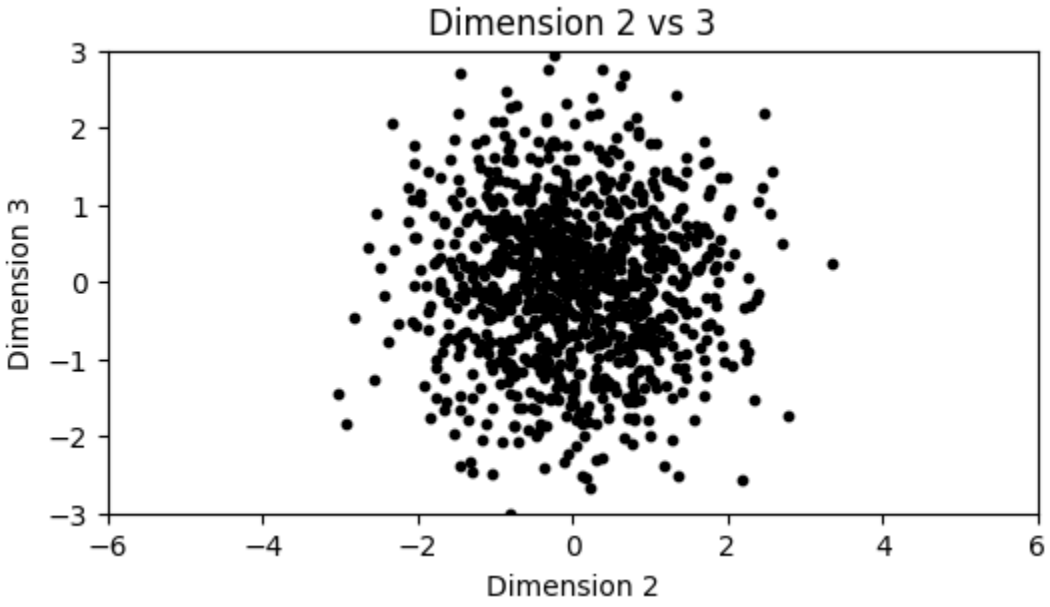
Now, let's plot first and second dimension.

In [12]:
```
#If we plot in any two dimensions not including the first one instead, we see only
_i = 1
_j = 2
plt.figure(figsize=(6, 3))
plt.scatter(X_1000d[:, _i], X_1000d[:, _j], s=10, c='k')
plt.axis([-6, 6, -3, 3])
plt.ylabel(f'Dimension {_j}')
plt.xlabel(f'Dimension {_i}')
plt.title(f'Dimension {_i} vs {_j}')
plt.show()
```



If we plot in any two dimensions not including the first one instead, we see only one cluster.

In [13]:
```
_i = 2
_j = 3
plt.figure(figsize=(6, 3))
plt.scatter(X_1000d[:, _i], X_1000d[:, _j], s=10, c='k')
plt.axis([-6, 6, -3, 3])
plt.ylabel(f'Dimension {_j}')
plt.xlabel(f'Dimension {_i}')
plt.title(f'Dimension {_i} vs {_j}')
plt.show()
```

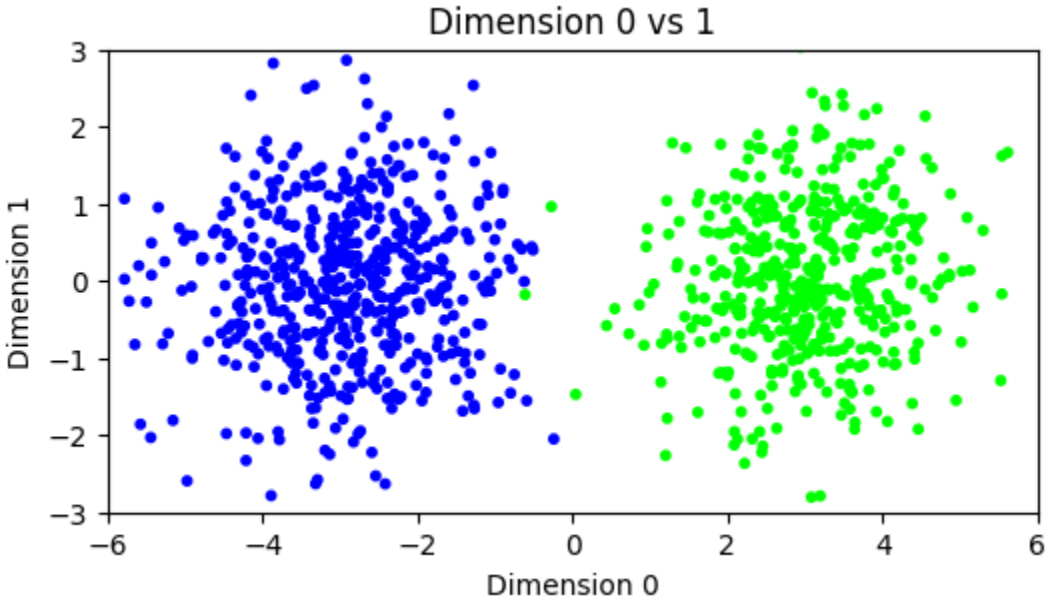


Use kmeans on the d = 100 dimensional data. Do we get the same results?

In [14]:
```
# What happens when we try to use kmeans on these columns?
assign_1000d = kmeans(rng, X_1000d, 2)
```

```
_i = 0
_j = 1
plt.figure(figsize=(6, 3))
plt.scatter(X_1000d[:, _i], X_1000d[:, _j], s=10, c=assign_1000d, cmap='brg')
plt.axis([-6, 6, -3, 3])
plt.ylabel(f'Dimension {_j}')
plt.xlabel(f'Dimension {_i}')
plt.title(f'Dimension {_i} vs {_j}')
plt.show()
```

```
1007651.8708298221
1006796.560118681
1005133.339458872
1000439.6375806011
999131.640064927
999114.1708464982
999109.872897009
999109.872897009
999109.872897009
999109.872897009
```



What happened? While the clusters are easy to tease apart if we know to look at the first coordinate only, in the full space the within-cluster and between-cluster distances become harder to distinguish: the noise overwhelms the signal.

As the dimension increases, the distributions of intra-cluster and inter-cluster distances overlap significantly and become more or less indistinguishable. That provides some insights into why clustering may fail here. Note that we used the same offset for all simulations. On the other hand, if the separation between the clusters is sufficiently large, one would expect clustering to work even in high dimension.

What is the **Curse of Dimensionality**?

As dimension increases:

- Volume grows exponentially
- Data become sparse
- Distance loses contrast

This is called the **curse of dimensionality**.

It is not a bug in the data. It is a consequence of geometry.

Curse of Dimensionality significantly impacts machine learning algorithms in various ways. It leads to increased computational complexity, longer training times, and higher resource requirements. Moreover, it escalates the risk of overfitting and spurious correlations, hindering the algorithms' ability to generalize well to unseen data.

In [15]:
```
def gmm2spherical_1(rng, d, n, phi0, phi1, mu0, sig0, mu1, sig1):
    phi, mu, sig = (np.stack((phi0, phi1)), np.stack((mu0, mu1)), np.stack((sig0, s
    X = np.zeros((n, d))
    component = rng.choice(2, size=n, p=phi)
    for _i in range(n):
        X[_i, :] = spherical_gaussian(rng, d, 1, mu[component[_i], :], sig[componen
    return (X, component)
```

In [16]:
```
_d, _n_1, _w_1 = (15, 1000, 3)

# Manually calling the logic of two_mixed_clusters but using gmm2spherical_1 to ret
_mu0 = np.hstack(([_w_1], np.zeros(_d - 1)))
_mu1 = np.hstack(([-_w_1], np.zeros(_d - 1)))

X_15d, component_15d = gmm2spherical_1(rng, _d, _n_1, 0.5, 0.5, _mu0, 1, _mu1, 1)
```

In [17]:
```
# What's component representing here?
print('Component shape:', component_15d.shape)

print('Unique Values in Component:', np.unique(component_15d))

component_15d[:10]
```

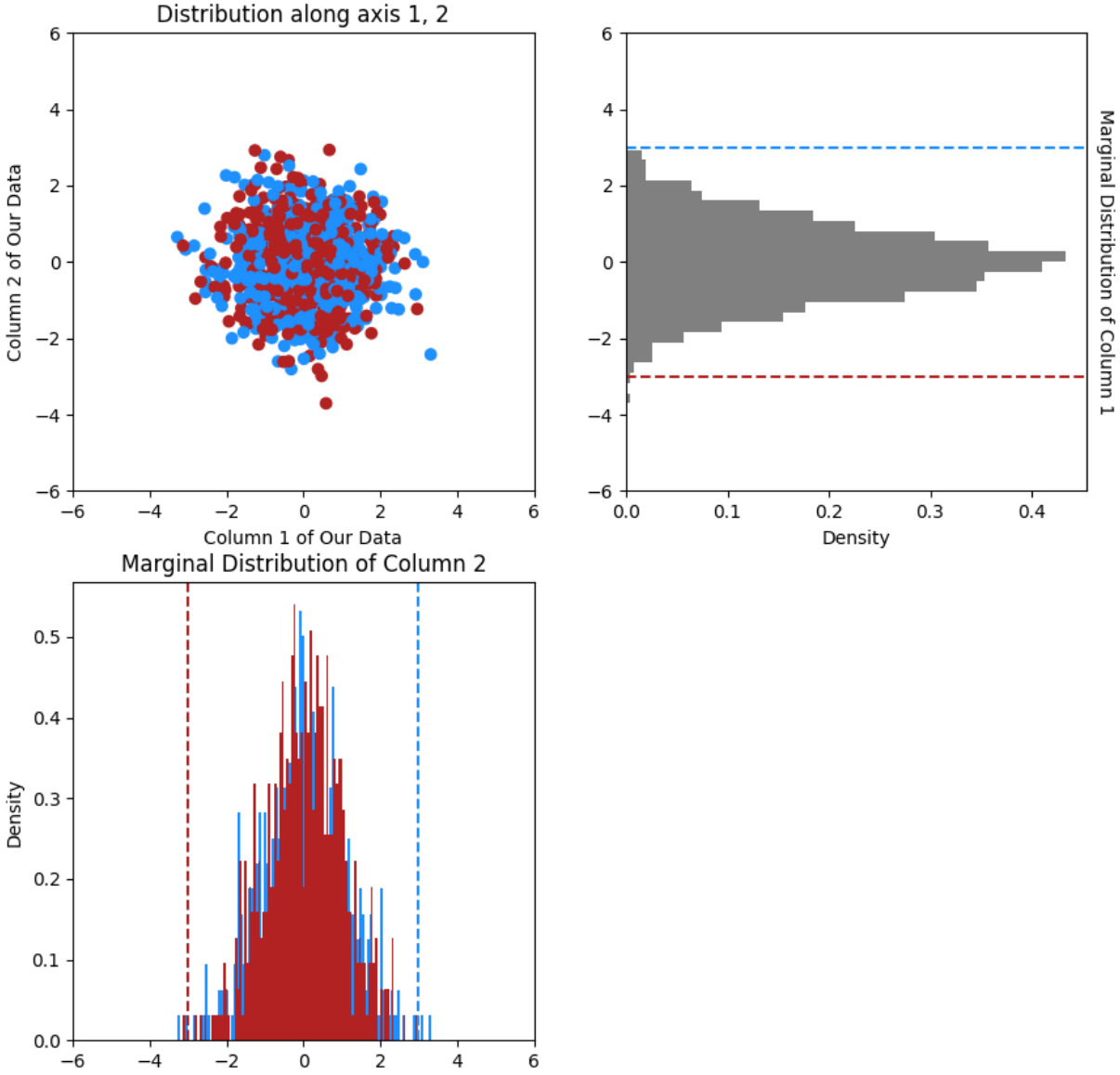
```
Component shape: (1000,)
Unique Values in Component: [0 1]
```

Out[17]:
```
array([0, 0, 1, 0, 1, 1, 0, 1, 0, 0])
```

In [18]:
```
_n_1 = X_15d.shape[0]
_w_1 = 3 # Hardcoded from previous cell logic
_i = 1
_j = 2
xlim_scaler = 2
legend = {0: 'dodgerblue', 1: 'firebrick'}
colors_15d = [legend[c] for c in component_15d]
comp1 = np.argwhere(component_15d == 0).flatten()
comp2 = np.argwhere(component_15d == 1).flatten()
_fig, ax = plt.subplots(2, 2, figsize=(10, 10))
```

```
ax[0, 0].scatter(X_15d[:, _i], X_15d[:, _j], color=colors_15d)
ax[0, 0].set_ylim(-xlim_scaler * _w_1, xlim_scaler * _w_1)
ax[0, 0].set_xlim(-xlim_scaler * _w_1, xlim_scaler * _w_1)
ax[0, 0].set_title(f'Distribution along axis {_i}, {_j}')
ax[0, 0].set_xlabel(f'Column {_i} of Our Data')
ax[0, 0].set_ylabel(f'Column {_j} of Our Data')
ax[0, 1].hist(X_15d[:, _j], orientation='horizontal', density=True, bins=25, color=
ax[0, 1].set_ylim(-xlim_scaler * _w_1, xlim_scaler * _w_1)
ax[0, 1].set_ylabel(f'Marginal Distribution of Column {_i}', rotation=270, labelpad
ax[0, 1].set_xlabel(f'Density')
ax[0, 1].yaxis.set_label_position('right')
ax[1, 0].hist(X_15d[comp1, _i], density=True, bins=np.min([100, round(_n_1 / 10)]),
ax[1, 0].hist(X_15d[comp2, _i], density=True, bins=np.min([100, round(_n_1 / 10)]),
ax[1, 0].set_title(f'Marginal Distribution of Column {_j}')
ax[1, 0].set_ylabel(f'Density')
ax[1, 0].set_xlim(-xlim_scaler * _w_1, xlim_scaler * _w_1)
ax[0, 1].axhline(y=_w_1, ls='--', color='dodgerblue')
ax[0, 1].axhline(y=-_w_1, ls='--', color='firebrick')
ax[1, 0].axvline(x=_w_1, ls='--', color='dodgerblue')
ax[1, 0].axvline(x=-_w_1, ls='--', color='firebrick')
_fig.delaxes(ax[1, 1])
_fig.show()
```

```
/tmp/ipykernel_6831/791997181.py:32: UserWarning: FigureCanvasAgg is non-interactiv
e, and thus cannot be shown
  _fig.show()
```


Distribution along axis 1, 2
Column 2 of Our Data
Column 1 of Our Data
Marginal Distribution of Column 1
Density
Marginal Distribution of Column 2
Density

In [19]:
```
### User Toggles
# Change the dimensions on the x,y,z axes
_x, _y, _z = (0, 1, 2)
_know_answer = True
# Toggle to change the color of the nodes
# from blue/red (True) vs grey (False)
if _know_answer:
    _color_nodes = colors_15d
else:
    _color_nodes = ['grey'] * X_15d.shape[0]
_background_color = '#ffffff'
_title = ''
_axis = dict(showbackground=True, showline=True, zeroline=True, showgrid=True, show
_layout = go.Layout(title=_title, width=1000, height=1000, scene=dict(xaxis=dict(_a
### Don't worry about code past this
_nodes = [dict(type='scatter3d', x=[X_15d[k][_x]], y=[X_15d[k][_y]], z=[X_15d[k][_z
_fig = go.Figure(data=_nodes, layout=_layout)
_fig.update_layout(scene=dict(xaxis_title=f'Dimension {_x}', yaxis_title=f'Dimensio
# define node dictionaries
_fig
```

From Geometry to Probability

So far, we have argued geometrically:

- Distances concentrate
- Nearest ≈ farthest

Now we switch viewpoints.

Instead of asking:

> "Where are the points?"

We ask:

> "How much variability is there, and how likely are large deviations?"

This is a probabilistic question.

Chebyshev's Inequality

Chebyshev's inequality gives a **guarantee**, not an exact probability.

It says:

> No matter what the distribution looks like, values are unlikely to be very far from the mean *unless variance is large*.

This bound becomes especially relevant in high dimensions.

Visual Proof of Chebyshev's Inequality

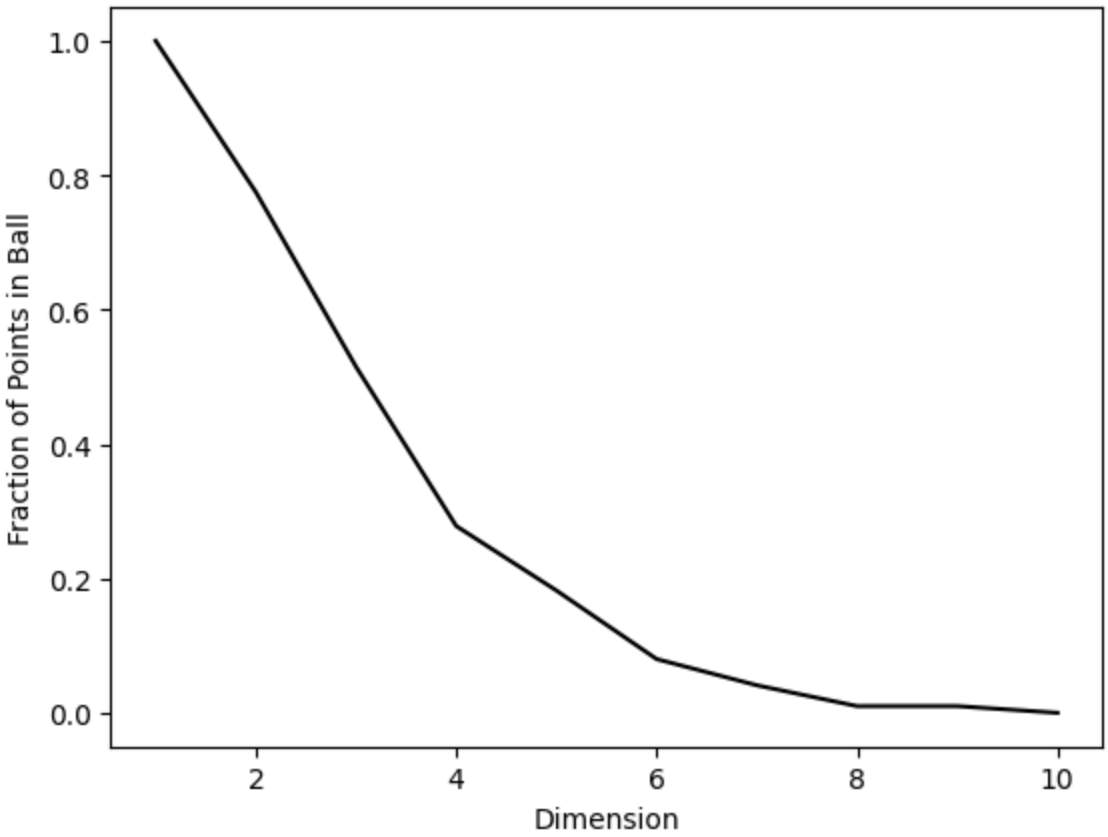
We can plot the frequency of landing in the inscribed d-ball B over number of dimensions and see that it rapidly converges to 0.

$$P(|X - E[X]| \geq \alpha) \leq \frac{Var[X]}{\alpha^2} = (\frac{\sigma_X}{\alpha})^2$$

Key points:

- This is an **upper bound**, not the true probability
- It applies to *any* distribution with finite variance
- Larger variance → weaker guarantees

In [20]:
```
dmax, n_cheby = (10, 1000)
in_ball = np.zeros(dmax)
for _d in range(dmax):
    in_ball[_d] = np.mean([LA.norm(rng.random(_d + 1) - 1 / 2) < 1 / 2 for _ in ran
plt.plot(np.arange(1, dmax + 1), in_ball, c='k')
plt.xlabel('Dimension')
plt.ylabel('Fraction of Points in Ball')
plt.show()
```



How to Read the Above Plot

This plot does **not** show:

- The true distribution
- Exact probabilities

It **does** show:

- How fast the bound decays
- Why variance dominates behavior
- Why high-dimensional distances concentrate

One-Sentence Takeaway

As dimensionality increases, variance grows, and probabilistic guarantees become weak — making distance an unreliable notion of similarity.

Why Functions Matter Here

We are not writing functions for efficiency.

We are writing them to:

- Encapsulate behavior
- Change assumptions with minimal code changes
- Explore how outcomes depend on parameters

This mirrors how modeling choices work in practice.

What is a function and how to use it

- Why are functions useful?
- What is parameter? Optional vs Required
- What are default values for parameters?
- Why is it important that parameters have type restraints? (input '1' vs 1)
- Printing something within a function vs returning a value from a function
- If/Elif/Else statements
- What are f-strings?
- How to handle returning multiple values from a function (or returning a list or tuple)?
  - What if you only want one of the values?
  - Does order matter?

In [25]:
```python
# Define a function that performs basic math operations
def basic_math_operations(a, b, operation='add', return_absolute=False):
    """
    Performs a basic mathematical operation on two numbers.

    Parameters:
    a (float): Required. The first number.
    b (float): Required. The second number.
    operation (str, optional): The mathematical operation to perform. Options: "add
    return_absolute (bool, optional): If True, returns the absolute value of the re

    Returns:
    tuple:
        - result (float or str): The computed result, or an error message if divisi
        - operation_used (str): The operation that was performed.
    """
    if operation == 'add':
        result = a + b  # Perform the selected operation
    elif operation == 'subtract':
        result = a - b
    elif operation == 'multiply':
        result = a * b
    elif operation == 'divide':
        result = a / b if b != 0 else 'Error: Division by zero'
    else:
        return ('Error: Invalid operation', operation)  # Handle division by zero
    if return_absolute and isinstance(result, (int, float)):
        result = abs(result)
    return result  # Return an error if an invalid operation is passed
```

```
# Examples
# _res1, _op1 = basic_math_operations(5, 3)
# print(f'Operation: {_op1}, Result: {_res1}')
```

In [26]:
```
# How do we use the output?:
# _res1, _op1 = basic_math_operations(5, 3)
# print(_res1)
# Previous example - Performing addition (default)
# print(_op1)
# the last line of the basic_math_operations function: return result, operation
# _res2 = basic_math_operations(5, 3)  #result
# The below code is incorrect for our use-case - it returns a tuple with 2 values i
# In the first addition example, you are assigning each value returned from the fun
# This means that (var1, var2) takes on the values of (8, 'add') respectively
# print(_res2)  #operation
```

Instructions

Modify the function so that:

- The window size is a parameter
- The subtraction is optional
- The function returns a single numeric value

Do **not** print intermediate values.

In [27]:
```
print(basic_math_operations(10, 4, operation='subtract'))
```

6